

جامعة سيدي محمد بن عبد الله بفاس
+08.000 0000 0000 0000 0000 0000 0000 0000
UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH DE FES



كلية العلوم والتقنيات فاس
+08.000 0000 0000 0000 0000 0000 0000 0000
FACULTÉ DES SCIENCES ET TECHNIQUES DE FES

Filière Master : Ingénierie Electrique et Systèmes Embarqués (IESE)

Technologies Web

JavaScript

Pr. Abdelali Boushaba

Année universitaire : 2025-2026

Chapitre 4 :Le langage JavaScript

Plan du chapitre 4

- Introduction
- Intégration de code JavaScript dans HTML
- Types de données
- Les Tableaux
- Les chaînes de caractères
- Les fonctions
- Création d'objet
- DOM
- Les événements
- Validation des données

Introduction à JavaScript

■ Définition

JavaScript est un langage de script interactives côté client qui peut être inséré dans des pages HTML et être interprété par les navigateurs Web.

- JavaScript permet d'accéder aux objets du navigateur via l'interface DOM (*Document Object Model*)

JavaScript est inspiré du langage Java

Comparaison entre JavaScript et Java

JavaScript	Java
Interprété	Compilé
Langage orienté objet basé sur la notion de prototypes	Langage de programmation purement orienté objets
Code visible	Code invisible
Typage faible	Typage fort
Débogage difficile	Débogage par des IDE (plusieurs IDE)

Intégration de code JavaScript dans HTML

Méthode1: Code JavaScript intégré au document html

Exemple: test

```
<HTML>
  <HEAD>
    <TITLE>Exemple 1</TITLE>
    <SCRIPT TYPE="text/javascript">
      // Du code Javascript
      alert("Bonjour Master IESE !"); // affiche une boîte de dialogue
    </SCRIPT>
    <NOSCRIPT><P>Votre navigateur n'accepte pas le Javascript.</P></NOSCRIPT>
  </HEAD>
  <BODY>
    ...
  </BODY>
</HTML>
```

Cette page indique
Bonjour Master IESE!

OK

Remarque : L'élément NOSCRIPT permet de fournir un contenu de remplacement pour les navigateurs qui ne peuvent exécuter un script.

Master IESE

7

Intégration de JavaScript dans HTML

Méthode 2: Code JavaScript externe

Exemple:

```
<HTML>
  <HEAD>
    <TITLE>Exemple 3</TITLE>
    <SCRIPT TYPE="text/javascript" SRC="unFichier.js"></SCRIPT>
    //ou bien
    <SCRIPT LANGUAGE="javascript" SRC="unFichier.js"></SCRIPT>
  </HEAD>
  <BODY>
    .....
  </BODY>
</HTML>
```

Master IESE

8

Intégration de JavaScript dans HTML

Méthode 3 : Code JavaScript associé à un événement

Exemple 1: test

```
<HTML>
<HEAD>
<TITLE>Exemple 1</TITLE>
</HEAD>
<BODY>
  <a href="JavaScript:alert('Bonjour Master IESE !');"> cliquez ici </a>
  <!-- ou bien <a href="" onClick="alert('Bonjour Master IESE !');"> cliquez ici </a> -->
</BODY>
</HTML>
```

Exemple 2: test

```
<html>
  <head>
    <title> Exemple 2</title>
  </head>
  <body>
    <SCRIPT LANGUAGE="JavaScript">
      function unMessage() {
        alert("Bonjour Master IESE !");
      }
    </SCRIPT>
    <FORM ACTION="" METHOD="post"><P>
      <INPUT TYPE="BUTTON" VALUE="cliquez ici" onClick="unMessage()">
    </FORM>
  </body>
</html>
```

Master IESE

9

Types de variables JavaScripts

Les types primitifs

String : chaînes de caractères

Number: nombres entiers et réels

Boolean : les booléens (true, false)

Undefined: valeur prise par les variables déclarées non initialisées

Les types structurels

Date: pour représenter les dates

Array: pour représenter les tableaux

Object: tel que les enregistrements

Function

Master IESE

10

Déclaration de variables en JavaScript

Déclaration explicite:

var : mot clés utilisé pour déclarer une variable accessible depuis l'extérieur d'un bloc { ... }

let : mot clés utilisé pour déclarer une variable dont la portée est limitée à un bloc

const : mot clés utilisé pour déclarer une variable avec une valeur constante (non modifiable une fois affectée).

Syntaxe:

```
var nom_variable;
```

```
var nom_variable = une_valeur;
```

```
var nom_variable = new Type_de_la_variable;
```

Déclaration implicite:

sans utilisation du mot clés var

Exemples:

Déclaration des variables booléens:

```
var choix=new Boolean( true );
```

```
choix=new Boolean(1) ;
```

```
let choix= true
```

Déclaration des chaînes de caractères:

```
var chaine = "Bonjour";
```

Déclaration des nombres:

```
var n= 20; //un entier ;
```

```
let PI = 3.1415 ; //un nombre réel.
```

Déclaration de plusieurs variables:

```
var m = 2, chaine= "Bonjour Monde";
```

Déclaration sans initialisation:

La valeur d'une variable déclarée sans initialisation, est *undefined*

```
var x;      typeof x;  // Retourne "undefined"
```

Master IESE

Portée des variables en JavaScript

Portée du bloc

Une variable déclarée à l'intérieur d'un bloc { } avec **let** n'est pas accessible depuis l'extérieur du bloc.

Une variable déclarée à l'intérieur d'un bloc { } avec **var** est accessible depuis l'extérieur du bloc.

Exemple:

```
{ let x=2; var y=1.5; }
```

// x n'est pas accessible par contre y est accessibles depuis l'extérieur du bloc.

Portée locale

Les variables déclarées dans une fonction ne sont accessibles qu'à l'intérieur de cette fonction

Exemple:

```
function uneFonction() {
```

```
    var x = 10; let y =30; const z=20;
```

```
}
```

//x, y et z ne sont pas accessibles depuis l'extérieur de la fonction

Portée globale

Si une variable déclarée en dehors de toutes les fonctions, alors elle est accessible de n'importe quel endroit du programme

Master IESE

12

Eléments propres à JavaScript

Eléments JavaScript identiques à C/C++/Java

Opérateurs d'affectation / Opérateurs arithmétiques / Opérateurs de comparaison / Structures de Contrôle et Boucles / Commentaires

Eléments propres à JavaScript

Opérateurs arithmétique JavaScript

- ****** puissance ($x^{**}2$: qui signifie x à la puissance de 2)

Opérateur d'affectation:

- ****=** ($x^{**}=y \Leftrightarrow x = x^{**}y$ qui signifie $x = x$ à la puissance y)

Opérateurs de comparaison

- **===** comparaison d'égalité en terme de valeur et de type : `var x = 10; var y = "10"; var z=(x===y); //z=false`
- **!==** comparaison de différence en terme de valeur et de type : `var x = 10; var y = "10"; var z=(x!==y); //z=true`
- **==** comparaison de la valeur : `var x = 10; var y = "10"; var z=(x==y); //z=true`
- **!=** comparaison de la valeur : `var x = 10; var y = "10"; var z=(x!=y); //z=false`

Structures de Contrôle et Boucles

```
if (condition(s)){  
    <instruction(s) 1>  
}  
else {  
    <instruction(s) 2>  
}
```

```
while (condition(s)){  
    <instruction(s) >  
}
```

```
do{  
    <instruction(s) >  
} while (condition(s));
```

```
switch (expression)  
{  
    case Val_1 : instruction(s)_1  
                break ;  
    case Val_2 : instruction(s)_2  
                break ;  
    case Val_... : instruction(s)_N  
                break ;  
    default : <instruction(s) par défaut>  
}
```

```
for (inst1 ; inst2 ; inst3){  
    <instruction(s) >  
}
```

```
for (var x of objet){ //parcourt les valeurs  
    <instruction(s) >  
}
```

```
for (var x in objet){ //parcourt les index  
    <instruction(s) >  
}
```

break; //interrompt le déroulement d'une boucle
continue; //passer directement à l'itération suivante sans exécuter les autres instructions d'une boucle

Les Tableaux en JavaScript

Création d'un tableau:

```
var T1 = new Array(val1, val2, ...);
var T2 = [val1, val2, ...];
var T3 = new Array();
```

Accès à un élément d'indice $i \geq 0$ d'un tableau

```
x = T[i];
```

Propriétés

Taille d'un tableau: T.length

Méthodes

push(elt_ajouter_debut), unshift(elt_ajouter_fin), shift() et pop() retirent respectivement le premier et le dernier élément, reverse(), sort(), ...

Parcours

for, for..of, for..in, while, do-while

Exemple : Création d'un tableau hétérogène:

```
var T = new Array(10, true, "bonjour");
T[0] = '20';
var chaine = "";
for (var i=0; i < T.length; i++){
    chaine += T[i] + ' '
}
alert(chaine); // affiche : 20 true bonjour
chaine = "";
T.reverse();
for (var x of T){
    chaine += x + ' '
}
alert(chaine); // affiche : bonjour true 20
```

Les chaînes de caractères en JavaScript

Création

```
var chaine1 = new String('Bonjour');
var chaine2 = 'Bonjour';
chaine2 = " IESE";
```

Concaténation '+'

```
var chaine3 = chaine1 + chaine2;
alert(chaine); // affiche : Bonjour IESE
```

Propriétés

Taille d'une chaîne de caractères: length

Méthodes

charAt(int i) : retourne le ième caractère
 substring(int i, int j) : retourne la chaîne de caractères comprise entre le ième caractère (inclus) et le jème caractère (exclus)
 toLowerCase : convertit une chaîne en minuscule
 toUpperCase : convertit une chaîne en majuscule
 parseInt : conversion d'une chaîne vers un entier
 parseFloat : conversion d'une chaîne vers un float
 T = ch.split('sep') découper une chaîne de caractères ch contenant le séparateur sep en tableau T
 chaine = T.join('sep') rassembler les éléments d'un tableau T en chaîne ch contenant le séparateur 'sep'

Méthodes (suite)

chaine.indexOf(sous_chaine, [pos]) : retourne l'indice de la 1^{ère} occurrence de la sous chaîne «sous_chaine» dans la chaîne «chaine»

chaine.lastIndexOf(sous_chaine, [pos]) : retourne l'indice de la dernière occurrence de la sous chaîne «sous_chaine» dans la chaîne «chaine»

Le paramètre pos est optionnel, il permet d'effectuer la recherche :

- à partir de la position d'indice «pos» dans le cas de la méthode indexOf

- jusqu'à la position «pos» dans le cas de la méthode lastIndexOf

Si sous chaîne «sous_chaine» n'est pas trouvée, les 2 fonctions renvoient -1

Autres Méthodes : bold() italics() fontcolor() fontsize() small() big()

Conversion de type en JavaScript

- La méthode *Number* permet de convertir les chaînes et dates en nombres

`Number("12.5")` // retourne 12.5

`Number("")` // retourne 0

`Number(" ")` // retourne 0

`Number("20 2024")` // retourne NaN

`d=new Date(); Number(d) ⇔ d.getTime();` // retourne number

- La méthode *String* (ou *toString*) permet de convertir des nombres et dates en chaînes

`String(x)` // retourne une chaîne de caractères des chiffres de la variable x

`String(2024)` // retourne la chaîne "2024"

`String(11+4)` // retourne la chaîne "15"

`String( Date() )` // retourne une chaîne représentant la date actuelle complète

Qui est équivalent à:

`x.toString()` ;

`(2024).toString()` ;

`(11+4).toString()` ;

`Date().toString`

Master IESE

17

Méthode de manipulation des variables de type Date

Méthode	Description
<code>getDate()</code>	Retourne le numéro du jour (1-31)
<code>getDay()</code>	Retourne un entier entre 0 et 6, correspondant aux jours de la semaine (dimanche = 0, lundi = 1, etc.).
<code>getMonth()</code>	Retourne le numéro du mois (0-11)
<code>getFullYear()</code>	Retourne l'année en 4 chiffres (yyyy)
<code>getHours()</code>	Retourne l'heure (0-23)
<code>getMinutes()</code>	Retourne les minutes (0-59)
<code>getSeconds()</code>	Retourne les secondes (0-59)
<code>getTime()</code>	Retourne les secondes depuis 1970

Exemple:

`d=new Date();`

`var n = d.getDate();`

Master IESE

18

L'objet Math

Méthodes	Description
abs()	Retourne la valeur absolue du nombre donné (exemple: x = -1; Math.abs(x); //retourne 1)
sqrt()	Retourne la racine carrée du nombre donné
exp()	Retourne la forme exponentielle du nombre donné
max(), min()	Retourne respectivement la valeur maximale et minimale des nombres donnés
pow()	Retourne la valeur de la base à la puissance de l'exposant
sin(), cos()	Retourne respectivement le sinus, le cosinus et la tangente du nombre donné
Acos(), asin(), atan()	Retourne respectivement l'arc cosinus, l'arc sinus et l'arc tangente du nombre donné en radians
ceil()	Retourne une plus petite valeur entière, supérieure ou égale au nombre donné
floor()	Retourne la plus grande valeur entière, inférieure ou égale au nombre donné
log()	Retourne le logarithme népérien d'un nombre
random()	Retourne un nombre aléatoire compris entre 0 et 1
round()	Retourne la valeur entière la plus proche du nombre donné
trunc()	Retourne une partie entière du nombre donné

Les fonctions en JavaScript

➤ Syntaxe des fonctions en JavaScript:

```
function nomFonction (param1, param2, param3, ...)
{
    <instruction(s)>;
    [return (valeur ou expression);]
}
```

➤ Appel des fonctions:

```
nomFonction (p1,p2,p3, ...);
```

➤ Exemple:

```
let s = somme (10, 20);
function somme (a, b)
{
    return a+b;
}
```


setTimeout() et clearTimeout()

- La méthode `setTimeout()` permet de déclencher une fonction après l'expiration d'un délai.

Syntaxe:

```
compteur = setTimeout( "nomFonction()", délai_en_milliseconde);
```

Exemple: lancer la fonction `demarrer()` après 10 secondes.

```
compteur = setTimeout("demarrer()",10000);
```

- La méthode `clearTimeout()` permet d'arrêter le temporisateur avant l'expiration du délai fixé.

Syntaxe:

```
clearTimeout(compteur);
```

Fonctions de lecture et d'écriture

- `document.write()` : Permet d'écrire dans le document HTML
- `window.alert()` : Permet d'écrire dans une boîte d'alerte
- `Console.log()` : Permet d'écrire dans la console du navigateur
- `window.print()` : Permet d'imprimer le contenu de la fenêtre en cours
- `prompt()` : Permet d'afficher une boîte de dialogue avec une zone de saisie, elle retourne la valeur saisie après validation, sinon elle retourne null.
- `getElementById()` : permet de récupérer les données de n'importe quel élément du document HTML

Exemple 1 : test

```
<html>
<body>
<script>
  var nom=prompt("Quel est votre nom? :");
  document.write(nom);
</script>
</body>
</html>
```

Exemple 2 : test

```
<html>
<body>
<form>
  Quel est votre nom? :<input type="text" id="nom">
  <button onclick = "alert(document.getElementById('nom').value);"> Afficher </button>
</form>
</body>
</html>
```

Exercice

- ✎ Ecrire un programme JavaScript qui :

demande à l'utilisateur de saisir des noms et de les stockés dans un tableau T.

- ✎ La saisie se termine par une clique sur ok sans rien saisir.

- ✎ rassemble les éléments du tableau T en chaîne ch on les séparant par le caractère '-'

- ✎ affiche le contenu de la chaîne ch

- ✎ NB. Utiliser les méthodes : prompt, push, alert et join.

Solution : test

```
<HTML>
  <HEAD>
    <TITLE>Tableau JavaScript</TITLE>
  </HEAD>
  <BODY>
    <script>
      var T = [];          //ou var T= new Array();
      var nom="";          //ou var nom= new String();
      do {
        nom = prompt("Entrez un nom :");
        if (nom!="")
          T.push(nom);
      }while(nom!="");
      var ch = T.join('-');
      alert(ch);
    </script>
  </BODY>
</HTML>
```


Création d'objet en JavaScript

Méthode 1 : Création d'objet avec syntaxe littérale

```
var unObjet = {  
    attribut1: valeur1,  
    attribut2: valeur2,  
    ...  
    methode1(...) {...},  
    methode2(...) {...}  
    ...  
};
```

Méthode 2 : Création d'objet avec constructeur

```
function unConstructeur(param1, param2, ...){  
    this.Attribut1= param1;  
    this.attribut2=param2;  
    ...  
    this.methode1 = function() {...}  
    this.methode2= function() {...}  
    ...  
};
```

Exemple : Création d'objet avec syntaxe littérale

```
var etudiant= {  
    nom : "Alami",  
    prenom : "Ali",  
    note1 : 10,  
    note2 : 15,  
    moyenne(){ return (this.note1+this.note2)/2; },  
};  
alert("la moyenne de "+ etudiant.nom + " " + etudiant.prenom  
+"est:" + etudiant.moyenne() );
```

Exemple : Création d'objet avec constructeur

```
function Etudiant (nm, pn, n1, n2) {  
    this.nom = nm;  
    this.prenom = pn;  
    this.note1 = n1;  
    this.note2 = n2;  
    this.moyenne(){ return (this.note1+this.note2)/2; },  
};  
var etudiant = new Etudiant ("Kamali", "Said", 13, 14);  
alert("la moyenne de "+ etudiant.nom + " " + etudiant.prenom  
+"est:" + etudiant.moyenne() );
```

Master IESE

25

DOM

DOM (Document Object Model) est une interface de programmation (API) JavaScript qui permet de manipuler les documents HTML et XML.

DOM représente la structure d'un document sous forme d'arbre, où chaque élément et contenu textuel est un "nœud".

DOM offre un ensemble de méthodes permettant d'interagir avec ces nœuds pour gérer (Modifier/ Ajouter/ Supprimer):

- Les éléments HTML de la page Web
- Les attributs des éléments HTML
- Les styles CSS
- Les événements dans un document HTML

Master IESE

26

DOM

- Dans un document HTML, on peut accéder aux objets (ou aux attributs et méthodes) de la manière suivante :

```
// Accès à une propriété de l'objet button
window.document.form.button.value = "Envoyer";
// Accès à une méthode de l'objet button
window.document.form.button.focus();
// Accès à une méthode de l'objet window
window.alert("Hello world");
```

Remarque : l'objet *window* est souvent facultatif. On ne doit pas forcément préciser son nom pour utiliser ses méthodes ou ses propriétés.

DOM

- L'objet *document* est un sous-objet de *window*. L'objet *document* possède trois méthodes principales pour accéder aux éléments d'un document HTML:

- getElementById('val_id') : retourne un seul élément

- getElementsByTagName('nom_balise') : retourne les éléments sous forme de tableau

- getElementsByName('nom_elt_form') : retourne les éléments sous forme de tableau

- Il est recommandé d'utiliser la méthode **getElementById()** pour accéder aux objets par leur identifiant (attribut ID de l'élément HTML) :

```
var bouton = document.getElementById('id_bouton');
bouton.click(); // pour simuler un clic de souris sur ce bouton
```


L'Arbre d'objets DOM

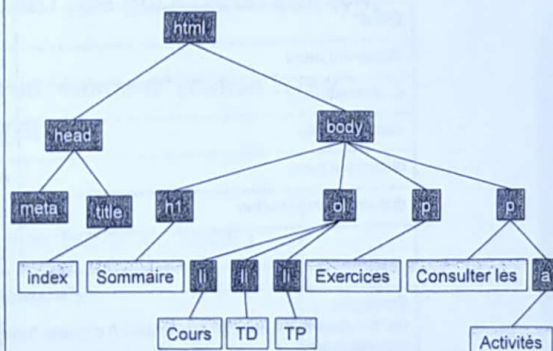
- DOM représente un document HTML de manière hiérarchique sous forme d'un arbre d'objets (les nœuds)
- L'objet le plus haut dans la hiérarchie est l'objet *window* (fenêtre).
- Dans l'objet *window* il y'a l'objet *document*.
- Dans l'objet *document* on peut trouver l'élément HTML *Form*, et dans *Form* on peut trouver l'élément *button* et ainsi de suite.
- La modification du document HTML à l'aide du DOM consiste alors à modifier, ajouter ou supprimer des nœuds de l'arbre

Exemple:

```
window.document.form.button.value = "OK"; // Accès à une propriété de l'objet button
```

L'Arbre d'objets DOM :Exemple

```
<html>
  <head>
    <meta charset="utf8">
    <title>Index</title>
  </head>
  <body>
    <h1>Sommaire</h1>
    <ol>
      <li>Cours</li>
      <li>TD</li>
      <li>TP</li>
    </ol>
    <p>Exercices</p>
    <p>
      Consulter les <a href="www.activites.com">Activités</a>
    </p>
  </body>
</html>
```



L'objet «document»

- L'objet *document* correspond à l'élément <html> de la page Web
C'est la racine de l'arbre DOM
- L'objet *document* dispose des propriétés *head*, *body*, *title* qui permettent d'accéder respectivement aux éléments <head>, <body> et <title> de la page web
- L'objet *document* dispose d'un ensemble de méthodes et d'objet permettant d'accéder et de manipuler le code html

L'objet «document»

■ Quelques objets:

Objet	Description
document.head	Accès à l'objet head dans le DOM
document.body	Accès à l'objet body dans le DOM
document.title	Accès à l'objet title dans le DOM
document.form	Accès à l'objet form dans le DOM
document.form.button	Accès à l'objet button dans le DOM
...	...

Exemple:

```
var h = document.head; // La variable h contient l'objet head du DOM
console.log(h);
var b = document.body; // La variable b contient l'objet body du DOM
console.log(b);
var t = document.title; // La variable t contient l'objet title du DOM
console.log(t);
var bt = document.form.button; // La variable bt contient l'objet button du DOM
console.log(bt);
bt.value = "Annuler" //Accès à une propriété de l'objet button
bt.button.focus()=uneFonction; // Accès à une méthode de l'objet button
```

L'objet «document»

Les méthodes de recherche d'éléments html:

Méthode	Description
document.getElementById (val_id)	Retourne un élément par la valeur de l'attribut id
document.getElementsByTagName (nom_balise)	Retourne les éléments par nom de balise sous forme d'un objet HTMLCollection
document.getElementsByClassName (nom_classe)	Retourne les éléments par nom de classe CSS sous forme d'un objet HTMLCollection

Les méthodes de recherche d'éléments html (par leur sélecteur CSS) :

Méthode	Description
document.querySelector(selecteur_CSS)	Retourne le premier élément qui correspond à un ou plusieurs sélecteurs CSS spécifiés. Exemple: Retourner l'élément dont l'attribut id="id1": document.querySelector("#id1")
document.querySelectorAll(selecteur_CSS)	Retourne tous les éléments qui correspondent à un ou plusieurs sélecteurs CSS spécifiés.

L'objet «document» : Types de noeuds du DOM

- Chaque objet du DOM a une propriété `nodeType` qui indique son type
- La valeur de cette propriété est:
 - `document.ELEMENT_NODE` pour un nœud "élément" (balise HTML)
 - `document.TEXT_NODE` pour un nœud textuel

```
Exemple:
if (document.body.nodeType === document.ELEMENT_NODE )
{
    console.log("Body est un nœud élément");
}
else
{
    console.log("Body est un nœud textuel");
}
//Body est un nœud élément
```


L'objet «document» : Navigation entre les nœuds de l'arborescence DOM

Navigation dans les nœuds enfants

- firstChild : Renvoie le premier **enfant** de l'élément.
- firstElementChild : Renvoie le premier **élément enfant** du parent.
- lastChild : Renvoie le dernier **enfant** de l'élément.
- lastElementChild : Retourne le dernier **élément enfant** du parent.
- childNodes : Renvoie tous les **enfants** de l'élément sous forme d'une collection.
- children : Renvoie tous les **éléments enfants** de l'élément sous forme d'une collection.

Navigation dans les nœuds parents

- parentNode : Renvoie le nœud parent de l'élément.
- parentElement : Renvoie l'élément parent de l'élément.

Navigation dans les nœuds frères

- nextSibling : Renvoie le **nœud frère** correspondant au prochain enfant du parent.
- nextElementSibling : Renvoie l'**élément frère** correspondant au prochain enfant de son parent.
- previousSibling : Renvoie le **nœud frère** qui est un enfant précédent de son parent.
- previousElementSibling : Renvoie l'**élément frère** qui est un enfant précédent de son parent.

L'objet «document»

Exemple : test

```
<html>
  <head>
    <title>Index</title>
  </head>
  <body>
    <h1>Sommaire</h1>
    <ol>
      <li>Cours</li>
      <li>TD</li>
      <li>TP</li>
    </ol>
    <p id="p1">Exercices</p>
    <p id="p2">
      Consulter les <a href="www.activites.com">
        Activités</a>
    </p>
```

Exemple (suite)

```
<script>
  let p = document.getElementById("p1");
  p.style.color="red";
  document.getElementById("p2").style.color="yellow";
  let list = document.getElementsByTagName("li");
  for (let i=0; i<list.length; i++)
    list[i].style.color="green";
  for (let elt of document.getElementsByTagName("li"))
    elt.style.color="red";
</script>
</body>
</html>
```


L'objet «document»

Les méthodes d'ajout et suppression d'éléments html:

Méthode	Description
document.createElement(un_element)	Créer un élément HTML
Node.remove()	Supprimer le nœud node
node.removeChild(un_element)	Supprimer l'élément enfant du nœud node
node.appendChild(un_element)	Ajouter un élément comme enfant au nœud node
node.replaceChild(un_nouveau_element, un_ancien_element)	Remplacer l'élément enfant du nœud node par un autre
document.write(text)	Écrire dans un document HTML
document.getElementById(d).onEvenement= uneFonction(){...}	Ajouter un événement « Evenement » à l'élément sélectionné (click, focus, submit ...)

Master IESE

37

Exemple : test

```
<html>
<head>
  <title> JavaScript </title>
</head>
<body onload="javascript:depart();">
  <p id="para">Bonjour</p>
  <script type="text/javascript">
    function depart () {
      var body = document.getElementsByTagName ("body");
      var texte1 = document.createTextNode ( " Programmation Web" ) ;
      body[0] .appendChild ( texte1 ) ;
      var paragraphe = document.getElementById ( "para" ) ;
      var texte2 = document.createTextNode ( " IESE" ) ;
      paragraphe.appendChild (texte2) ;
    }
  </script>
</body>
</html>
```

Master IESE

38

L'objet «document»

Propriétés des éléments DOM

Méthode	Description
un_element.innerHTML	Permet de récupérer tout le contenu HTML d'un élément du DOM
un_element.attribute	Permet de changer l'attribut d'un élément du DOM
un_element.style.property	Permet de changer le style d'un élément du DOM
un_element.textContent	Permet de récupérer tout le contenu textuel d'un élément du DOM, sans balises HTML
un_element.classList	Permet de récupérer la liste des classes d'un élément du DOM

Master I/SE

39

L'objet «document» : Propriété innerHTML

La propriété innerHTML permet de modifier le contenu d'un élément

Exemple: test

```
<body>
  <p id="p1"> Cours</p>
  <p id="p2"> IESE</p>
  <p id="p3"> Master</p>
  <script>
    let parag1 = document.getElementById("p1");
    let parag2= document.getElementById("p2");
    let parag3= document.getElementById("p3");
    parag1.innerHTML="Cours <b>Programmation Web</b>"
    parag2.innerHTML=parag3.innerHTML;
    parag3.innerHTML="<b>IESE</b>";
  </script>
</body>
```

Master I/SE

40

Les événements JavaScript

- Un événement est généralement associé à une action de l'utilisateur : appui sur une touche, clic ou déplacement de la souris, ...
- En HTML, il y a très peu d'événements qui sont gérés par défaut : clic sur un lien ou sur un bouton de formulaire.
- Javascript permet de gérer et contrôler les événements par des gestionnaires d'événements

Master IJSE

41

Les événements JavaScript

- Les trois méthodes principales pour ajouter un gestionnaire d'événements :
 - (1) Utilisation de l'Attribut `onEvenement` d'un élément HTML
 - (2) Utilisation de la Propriété `onEvenement` d'un élément (objet de l'arbre DOM)
 - (3) Utilisation de la Méthode `addEventListener()` d'un élément (objet de l'arbre DOM)

Master IJSE

42

(1) Utilisation de l'Attribut onEvenement d'un élément HTML

Syntaxe :

`onEvenement = "unefonction()"` ;

- **Evenement** : est le nom de l'évènement géré (par exemple : click, change, submit, etc.)
- `unefonction()` : est l'action qui sera automatiquement déclenchée lorsque l'évènement associé se produit.

Exemple:

```
<button onclick="alert('Bouton cliqué !')">Cliquez ici</button>
```

Master IESE

43

(2) Utilisation de la Propriété onEvenement d'un élément

Syntaxe :

`unElement.onEvenement = unefonction ;`

- `unElement` : est l'élément sujet de l'évènement (un bouton, un lien, un paragraphe, ...)
- **Evenement** : est le nom de l'évènement géré (par exemple : click, change, submit, etc.)
- `unefonction()` : est l'action qui sera automatiquement déclenchée lorsque l'évènement associé se produit.

Exemple:

```
<button id="unBouton">Cliquez ici</button>
<script>
  function message() { alert("Bonjour IESE!"); };
  let btn = document.getElementById("unBouton");
  btn.onclick = message;
</script>
```

Master IESE

44

(3) Utilisation de la Méthode `addEventListener()` d'un élément

Syntaxe :

- 1 `unElement.addEventListener('Evenement', uneFonction);`
- 2 `unElement` : est l'élément sujet de l'événement (un bouton, un lien, un paragraphe, ...)
- 3 `addEventListener` : est la méthode qui permet de spécifier un gestionnaire d'événements sur l'élément `unElement`
- 4 `Evenement` : est le nom de l'évènement géré (par exemple : `click`, `change`, `submit`, etc.)
- 5 `uneFonction()` : est l'action qui sera automatiquement déclenchée lorsque l'évènement associé se produit.

Exemple :

```
<button id="unBouton">Cliquez ici</button>
<script>
    function message() { alert("Bonjour IESE!"); };
    let btn = document.getElementById("unBouton");
    btn.addEventListener('click', message);
</script>
```

Master IESE

45

Les événements - Souris

- 1 Cas d'une action du bouton de la souris:

MouseDown : Cet événement est déclenché lorsque le bouton de la souris est enfoncé sur un élément

Mouseup : Cet événement est déclenché lorsque le bouton de la souris est relâché sur un élément

Click : Cet événement est déclenché lors d'un clic de la souris (enfoncé puis relâché) sur un élément

Dbclick : Cet événement est déclenché lors d'un double-clic de la souris sur un élément

- 2 Cas d'un déplacement de la souris

MouseOver : Cet événement est déclenché lorsque un utilisateur place le pointeur de la souris sur un élément

MouseOut : Cet événement est déclenché lorsque le pointeur de la souris sort d'un lien ou tout autre élément

MouseMove : Cet événement est déclenché lorsque la souris bouge sur un élément

Master IESE

46

Les événements - Clavier

- ⌨️ keydown : Cet événement est déclenché lorsque l'utilisateur enfonce une touche du clavier
- ⌨️ keyup : Cet événement est déclenché lorsque l'utilisateur relâche une touche du clavier
- ⌨️ keypress : Cet événement est déclenché lorsque l'utilisateur appuie (enfonce puis relâche) sur une touche du clavier

Exemple: test

```
<html>
<head>
<title> Exemple </title>
</head>
<body>

<input id="zoneTexte" type="text" onkeypress="changerClavier()" />
</form>
</form>
<p id="compteur"></p>
<script>
let n=0;
function changerClavier(){
n++;
document.getElementById("compteur").innerHTML = n;
}
</script>
</body>
</html>
```

Master I/EE

47

Les événements - Clavier

Exemple (version 2 : Propriété de l'événement d'un élément) : test

```
<html>
<head>
<title> Exemple 2</title>
</head>
<body>

<form>
</form>
<input id="zoneTexte" type="text" />
</form>
<p id="compteur"></p>
<script>
let n=0;
function changerClavier(){
n++;
document.getElementById("compteur").innerHTML = n;
}
var maZoneTexte = document.getElementById("zoneTexte");
maZoneTexte.onkeypress=changerClavier;
</script>
</body>
</html>
```

Master I/EE

48

Les événements - Formulaires

Cas 1 : Événements liés au formulaire

- 1 change
Cet événement est déclenché lorsque le contenu d'un élément est modifié
- 2 submit
Cet événement est déclenché lors de la soumission d'un formulaire
 - » **Remarque:** dans ce cas la fonction associée à cet événement doit retourner «true» ou «false» pour valider la soumission
- 3 reset
Cet événement est déclenché lors de la réinitialisation du formulaire

Cas 2 : Événements liés à la sélection

- 4 focus
Cet événement est déclenché lorsque l'élément récupère le focus (la zone d'entrée devient active)
- 5 blur
Cet événement est déclenché lorsque l'élément perd le focus
- 6 Select
Cet événement est déclenché lorsqu'une partie du contenu de l'élément est sélectionnée

Master I/EE

49

Les événements - Formulaires

Exemple : test

```
</html>
<head>
</head>
<body>

<form id="unFormulaire" >
  Saisir une note sur 20 : <input type="text" name="note" id="note"/>
  <input type="submit"/>
</form>
<script type="text/javascript">
  function verifier() {
    var uneNote= document.getElementById("note").value;
    if (uneNote== 0 && uneNote<= 20)
      return true;
    else{
      alert("La note doit être entre 0 et 20");
      return false;
    }
  }
  document.getElementById("unFormulaire").onsubmit=verifier;
</script>
</body>
</html>
```

Master I/EE

50

Les événements - Page

load

Cet événement est déclenché lors du chargement de la page web

- Utilisé dans la balise <body>

unload

Cet événement est déclenché lorsque la page est fermée

- Utilisé dans la balise <body>

error

Cet événement est déclenché suite à une erreur de chargement

- Utilisé dans la balise

abort

Cet événement est déclenché lorsque le chargement de la page est abandonné

- Utilisé dans la balise

Master I/SE

51

La validation des données

La validation des données permet de s'assurer que les données entrées par l'utilisateur est conforme aux données attendues.

Parmi les vérifications à faire :

- Remplissage de tous les champs obligatoires

- Saisie d'une date valide

- Saisie d'un texte dans un champ numérique

On distingue deux types de validation

- La validation côté serveur effectuée par un serveur Web, une fois que les données sont envoyées au serveur

- La validation côté client effectuée par un navigateur Web, avant que les données ne soient envoyée à un serveur Web.

Master I/SE

52

Validation d'un formulaire

- Exemple : Si un champ de formulaire est vide, la fonction affiche un message et renvoie false pour empêcher la soumission du formulaire

```
function validerForm(){  
    let valeur = document.forms.myForm.nom.value  
    if(valeur == ""){  
        alert ("Le champ Nom doit être saisi!");  
        return false;  
    }  
    else  
        return true ;  
}  
  
<form name="myForm" action="code.php" onsubmit="return validerForm()" method="post">  
Nom: <input type="text" name="nom">  
<input type="submit" value="Envoyer">  
</form>
```

Master IESE

53

Validation d'un formulaire

- La validation du formulaire HTML peut être effectuée automatiquement par le navigateur
- Si un champ de formulaire (nom) est vide, l'attribut *required* empêche la soumission du formulaire

```
<form name="myForm" action="code.php" method="post">  
Nom: <input type="text" name="nom" required >  
<input type="submit" value="Envoyer">  
</form>
```

Master IESE

54

Activité :

- ❏ Qu'est ce que JavaScript ?
- ❏ Quelle est la différence entre Java et JavaScript ?
- ❏ Quels sont les avantages de JavaScript ?
- ❏ Dans quelle balise HTML peut on placer le code JavaScript ?
- ❏ Est ce qu'un programme JavaScript crée un fichier compilé ?
- ❏ Le fichier externe de JavaScript doit il contenir la balise <script> ?
- ❏ Quel est le bon endroit pour insérer un code JavaScript ?
- ❏ Est ce que JavaScript peut accéder au système de fichiers ?

Master I/EESE

55

Activité :

- ❏ Qu'est ce que JavaScript
JavaScript est un langage de script côté client qui peut être inséré dans des pages HTML et être interprété par les navigateurs Web.
- ❏ Quelle est la différence entre Java et JavaScript

JavaScript	Java
✓Interprété	✓compilé
✓orienté prototypes	✓langage de programmation purement orienté objets
✓code visible	✓code invisible
✓typage faible	✓typage fort
✓Débogage difficile	✓Débogage par des IDE (plusieurs IDE)

Master I/EESE

56

Activité :

- 1. Quels sont les avantages de JavaScript
 - 1. JavaScript est un langage côté client.
 - 2. JavaScript est relativement rapide pour l'utilisateur final.
 - 3. JavaScript ajoute des fonctionnalités aux pages Web.
 - 4. JavaScript ne nécessite aucune compilation nécessaire.
 - 5. JavaScript est indépendant de la plateforme.
- 2. Dans quelle balise HTML plaçons nous le code JavaScript
 - 1. La balise <script>.

Master IESE

- 3. Est ce qu'un programme JavaScript crée un fichier compilé
 - 1. Non
- 4. Le fichier externe de JavaScript doit contenir la balise <script>
 - 1. Non.
- 5. Quel est le bon endroit pour insérer un code JavaScript
 - 1. Dans les sections < head > et <body>, ou dans un fichier
- 6. Est ce que JavaScript peut accéder au système de fichiers
 - 1. Non.

57